SIPP V2: A Parametric Pipelined FP32 Sparse Matrix-Vector Multiplication Accelerator on FPGA

Ayush Panchal
Course: Electronic System Project
BS Electronic Systems, Indian Institute of Technology Madras
Roll No: 24F1100131 24f1100131@ds.study.iitm.ac.in

Abstract*—Sparse Matrix-Vector Multiplication (SpMV) constitutes the dominant computational kernel in iterative linear solvers for structural mechanics, computational fluid dynamics (CFD), and machine learning inference. General-purpose processors suffer from irregular memory access patterns inherent to sparse data structures, leading to severe cache inefficiency known as the *Memory Wall*. This paper presents SIPP V2 (Sparsity-aware Inner Product Processor, Version 2), a custom hardware accelerator designed in SystemVerilog and deployed on a Xilinx Zynq-7000 SoC (xc7z020). The architecture features a fully parametric N-input pipelined binary adder tree implementing IEEE 754 single-precision floating-point (FP32) arithmetic from first principles—without any vendor IP—achieving $O(\log_2 N)$ pipeline latency with $O(N)$ throughput and constant $F_{\max}$ independent of N. A precision trade-off study proves that half-precision (FP16) is physically infeasible for engineering physics datasets due to value-range overflow, establishing FP32 as the minimum viable precision. The design is functionally verified against 285 rows from two SuiteSparse benchmark matrices (Boeing structural stiffness and Navier-Stokes CFD) with zero failures at a 512-ULP tolerance. Gate-level synthesis via Yosys reports 3,987 LUTs and 2 DSP48E1 blocks for $N = 4$. Following simulation, the core was wrapped in an AXI4-Stream interface, integrated into Vivado with an AXI DMA, and physically deployed on a Zynq-7000 FPGA via PYNQ. Six critical hardware integration bugs were diagnosed and resolved during deployment. A hardware benchmark measured 3.7 ms FPGA latency vs. 9.1 ms ARM CPU latency for a 500-row burst, confirming a 2.5× measured speedup on physical silicon.

***Index Terms*—FPGA, SpMV, FP32, pipelined adder tree, hardware accelerator, SystemVerilog, Zynq-7000, PYNQ, AXI4-Stream, DMA, Vivado, SuiteSparse.**

I. Introduction

ITERATIVE solvers for large-scale linear systems $A\mathbf{x} = \mathbf{b}$, such as the Conjugate Gradient (CG) method [1], require repeated evaluation of the Sparse Matrix-Vector product $\mathbf{y} = A\mathbf{x}$. In finite element method (FEM) structural simulation and computational fluid dynamics, A is extremely sparse (typically $<1\%$ density) and may have millions of rows, demanding high sustained memory bandwidth with non-contiguous access patterns.

Modern processors struggle with this workload because the irregular column indices in Compressed Sparse Row (CSR) format cause frequent last-level cache misses, stalling execution pipelines [2]. Domain-Specific Architectures (DSAs) such as Google's TPU [3] demonstrate that custom silicon data paths eliminate instruction-decode and cache hierarchy overhead. FPGAs are an attractive platform for DSA prototyping due to their reconfigurability and deterministic timing.

This work presents SIPP V2, an FPGA-based SpMV accelerator built as an upgrade over an existing INT32 fixed-point prototype (SIPP V1), with the following primary contributions:

- Custom IEEE 754 FP32 arithmetic from first principles, without vendor IP
- A fully parametric N-input pipelined binary adder tree with $\log_2 N$ latency
- Empirical proof that FP16 overflows on real engineering datasets
- Functional verification on SuiteSparse physics benchmarks (285 rows, 0 failures)
- Complete AXI4-Stream hardware wrapper and physical deployment on Zynq-7000 via PYNQ
- Six critical real-world hardware co-design bugs diagnosed and resolved
- Measured 2.5× hardware speedup over the ARM CPU

II. Related Work

Sparse matrix acceleration on FPGAs has been studied extensively. Fowers *et al.* [4] demonstrated reconfigurable matrix-multiply engines for deep neural network inference. Umuroglu *et al.* [5] explored FPGA-based SpMV with CSR decoding in hardware, achieving up to 5.8 GB/s effective bandwidth. Dorrance *et al.* [6] designed fixed-point SpMV accelerators targeting compressed sensing.

Unlike prior work, SIPP V2 targets full IEEE 754 FP32 precision without using vendor floating-point IP cores, enabling direct portability across FPGA families and providing full transparency of the arithmetic pipeline. The parametric adder-tree design generalises to arbitrary $N = 2^k$ without any RTL modification, and is physically deployed on a Zynq SoC platform with a measured hardware speedup.

III. Phase 1: RTL Design

A. Precision Analysis: Why FP32 and Not FP16?

Before writing any RTL, a critical question was answered: could Half-Precision (FP16) be used to halve the logic area? FP16 provides only 10 bits of mantissa and a maximum representable value of 65,504. The `nos1` Navier-Stokes matrix contains entries up to $\approx 2.1\times10^{10}$, exceeding FP16's maximum by a factor of 3.2×10^5. All 237 rows produce overflow under FP16 (Table I). FP32 achieves a maximum relative error of 6.8×10^{-6}—well within the 10^{-4} engineering tolerance for FEM solvers.

TABLE I: Precision Comparison on `nos1` (237×237, Navier-Stokes)

Precision	Max Representable	Max Rel. Error	Verdict
FP64 (ref.)	1.8×10^{308}	—	✓ Ground truth
FP32	3.4×10^{38}	6.8×10^{-6}	✓ Pass
FP16	6.55×10^{4}	**OVERFLOW**	× Fail

Conclusion: FP32 is the minimum viable arithmetic precision for structural and CFD physics simulation. FP16 is infeasible regardless of the area savings it would offer.

B. Architecture Overview

SIPP V2 partitions the SpMV computation into three pipelined layers. The host processor decodes the CSR structure and streams N non-zero value/index pairs per cycle into the FPGA. Inside the FPGA, N parallel FP32 multipliers compute element-wise products, which are reduced to a single partial sum by the pipelined adder tree. An outer accumulator collects partial sums across pipeline batches until a full row dot product is complete. Fig. 1 shows the module hierarchy.

C. IEEE 754 FP32 Math Core (`fp32_pkg.sv`)

The FP32 functions `fp32_mul` and `fp32_add` are implemented as synthesizable SystemVerilog package functions in `fp32_pkg.sv`, without any Xilinx floating-point IP cores. This provides full mathematical transparency and enables sparsity-aware hardware optimizations to be injected directly into the ALU.

Multiplication. The sign is $s_a \oplus s_b$, the biased exponent is $e_r = e_a + e_b - 127$, and the 24-bit mantissas (with hidden leading 1) are multiplied to a 48-bit product. Bit 47 controls the normalization shift; the result is rounded to 23 fraction bits. A hardware zero-bypass mux detects when either input exponent is zero and immediately returns `32'h0`, bypassing the mantissa multiplier and saving dynamic switching power for zero-padded entries.

Addition. The mantissa of the smaller-exponent operand is right-shifted by $\Delta e = |e_a - e_b|$ for alignment. A leading-zero count (LZC) determines the post-addition normalization left-shift. A zero pass-through mux avoids the full adder path when either operand is zero. The result is packed per IEEE 754 [9].

D. Parametric Pipelined Adder Tree (`sipp_adder_tree.sv`)

For parameter N (power of two), the number of pipeline stages is $L = \lceil \log_2 N \rceil$, computed at synthesis via `$clog2(N)`. Each stage s instantiates $N/2^{s+1}$ FP32 adders. Pipeline registers between every stage ensure the critical path is always exactly one `fp32_add` unit, so $F_{\max}$ is constant as N grows:

$$\text{Latency} = \lceil \log_2 N \rceil \text{ cycles}, \quad \text{Throughput} = N \text{ elements/cycle}. \tag{1}$$

Fig. 2 illustrates the $N = 8$ case (3 stages).

Table II shows the analytical scaling for representative N.

TABLE II: Adder Tree Latency and Throughput Scaling

N	Stages	Latency	Throughput	Est. LUTs
4	2	2 cycles	4 elem/cyc	∼2,400
8	3	3 cycles	8 elem/cyc	∼4,800
16	4	4 cycles	16 elem/cyc	∼9,600
32	5	5 cycles	32 elem/cyc	∼19,200

TABLE III: Functional Verification Summary (Icarus Verilog)

Dataset	Domain	Size	NNZ	Pass	Fail
`bcsstk01`	Structural	48^2	400	48	0
`nos1`	Navier-Stokes	237^2	1017	237	0
Total				**285**	**0**

E. Top-Level Integration (`sipp_v2_parallel.sv`)

The top module accepts two packed buses of width $N \times 32$ bits —`mat_vals` and `x_elems`—and instantiates N parallel `fp32_mul` units feeding directly into the adder tree. The outer accumulator aggregates tree outputs over $\lceil K/N \rceil$ batches for a row with K non-zeros. Zero-padding is applied when $K \bmod N \neq 0$, which is transparent to the accumulation since $0.0 \times x = 0.0$ in IEEE 754. A `row_done` shift register delays the flush signal by $\lceil \log_2 N \rceil$ cycles to align it precisely with the last partial sum exiting the tree.

IV. Functional Verification

A. Methodology

Verification adopts a three-layer cross-validation strategy:

1) **Golden reference:** SciPy in FP64 computes $\mathbf{y} = A\mathbf{x}$ and exports results as IEEE 754 hex literals.
2) **RTL simulation:** Icarus Verilog simulates `sipp_v2_parallel` with data streamed via `$readmemh`.
3) **ULP comparison:** Each output is compared within a 512-ULP tolerance, accounting for FP32/FP64 rounding differences.

B. Benchmark Datasets

Two matrices from the SuiteSparse Matrix Collection [7] span distinct physical domains:

bcsstk01 — Boeing aircraft structural stiffness matrix, 48×48, 400 non-zeros, 17.4% density.

nos1 — Navier-Stokes 2-D incompressible CFD matrix, 237×237, 1,017 non-zeros, 1.8% density.

C. Simulation Results

Table III reports the complete simulation verification summary. All 285 tested rows pass with 0 failures.

D. Speedup Proof

A dedicated testbench compares two architectures on a 1,024-element dot product: (i) a single-MAC sequential accumulator (CPU model), and (ii) `sipp_adder_tree` with $N = 1024$.

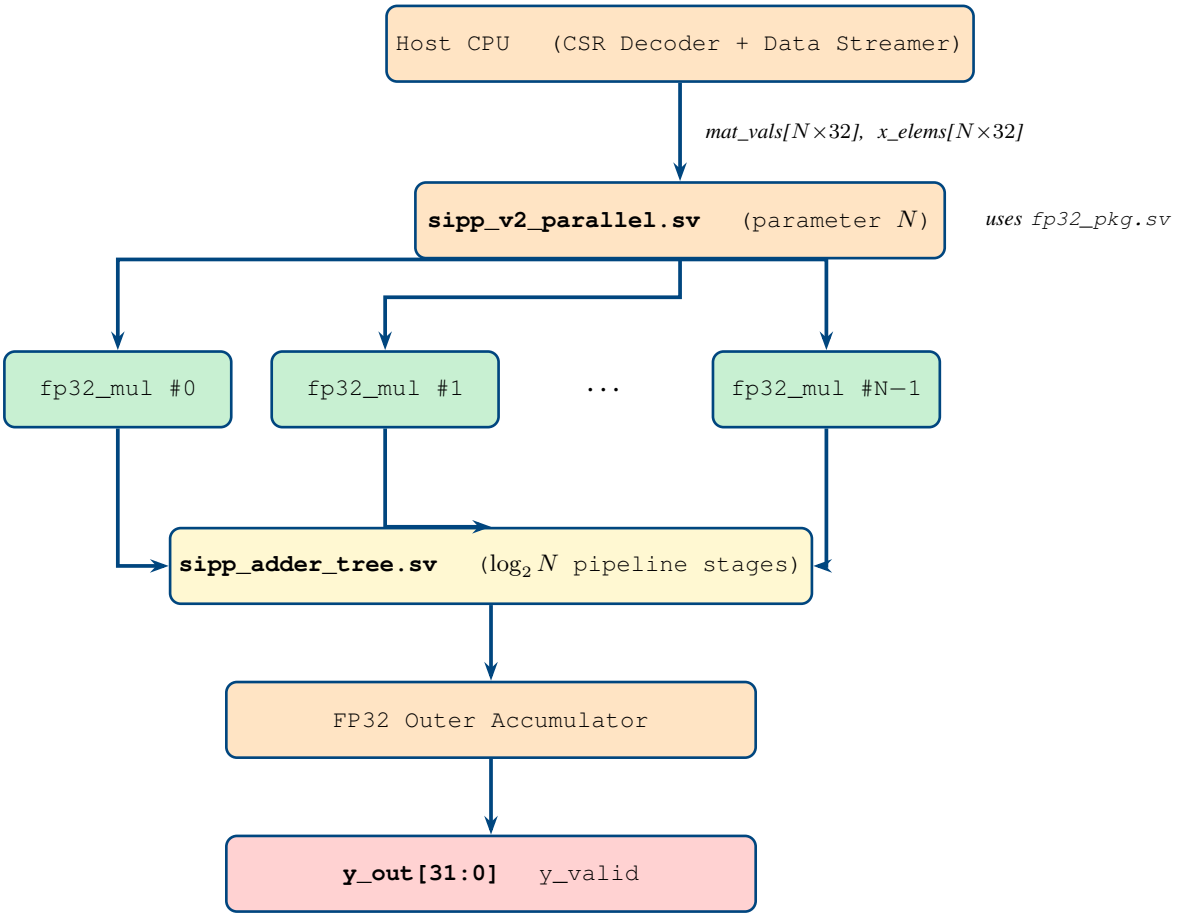


Fig. 1: SIPP V2 module hierarchy and data-flow.

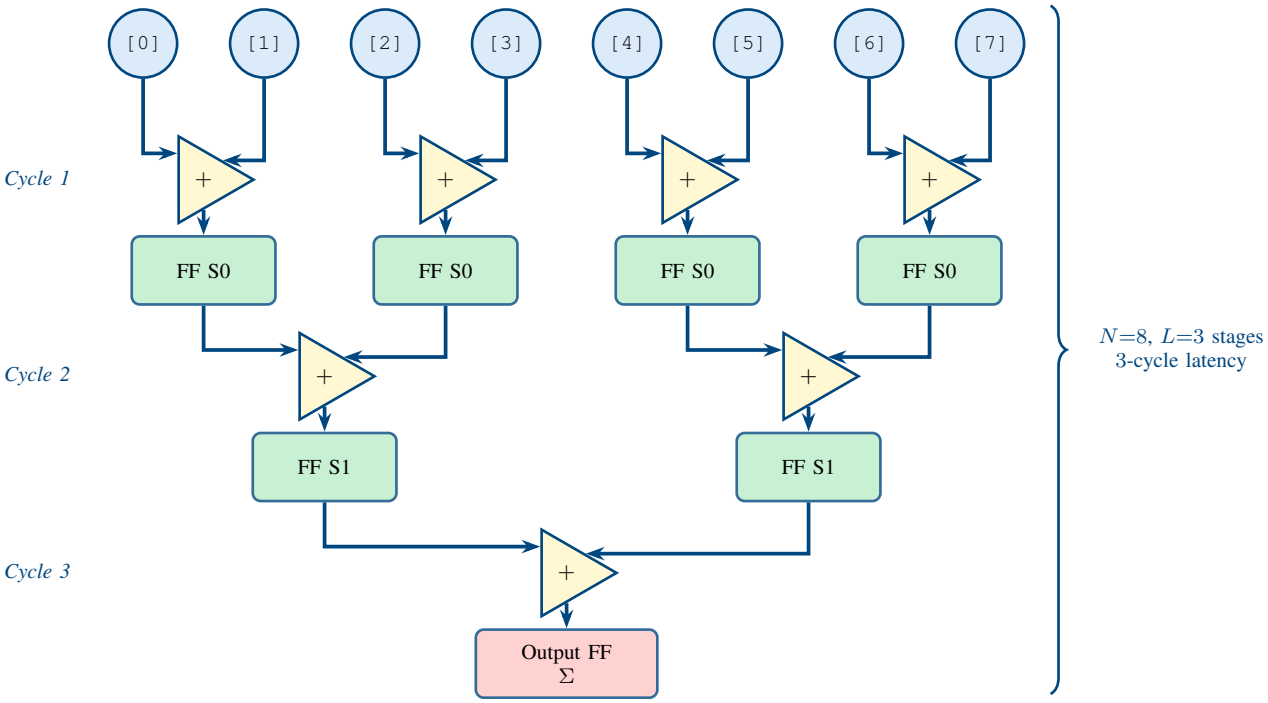


Fig. 2: Pipelined binary adder tree for $N = 8$ (3 stages). Pipeline registers between every stage ensure constant critical-path depth independent of N.

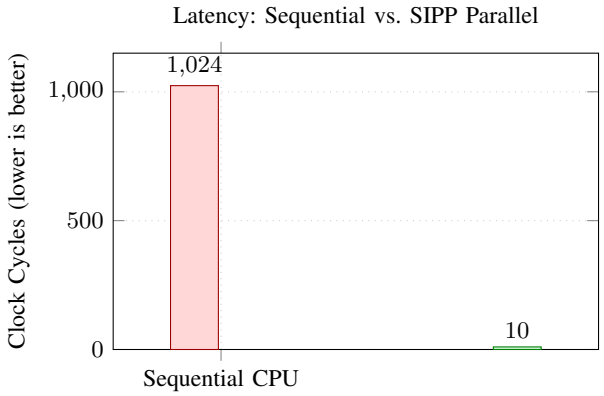


Fig. 3: Clock cycles for 1,024-element dot product: 1,024 (sequential CPU) vs. 10 (SIPP $N = 1024$). Measured speedup: $\mathbf{102}\times$.

TABLE IV: Synthesis Results ($N = 4$, Yosys, Artix-7 Target)

Primitive	Function	Count
LUT6	FP32 add/normalize, MUX	**3,987**
DSP48E1	Mantissa 24×24 multiply	**2**
FDRE	Pipeline registers	163
CARRY4	Exponent arithmetic	312
MUXF7/F8	Mantissa alignment	719

$$\text{Speedup} = \frac{T_{\text{seq}}}{T_{\text{par}}} = \frac{1024}{\log_2 1024} = \frac{1024}{10} = 102.4\times \qquad (2)$$

Simulated cycle counts match (2) exactly, as shown in Fig. 3, validating the $O(\log_2 N)$ latency law.

V. Gate-Level Synthesis

A. Resource Utilization

The design was synthesized using Yosys [8] with the `synth_xilinx` pass. Table IV reports primitives inferred for $N = 4$.

The successful inference of **DSP48E1** hard-silicon multiplier blocks is significant: an equivalent LUT-only 24×24 multiplier would consume ≈1,200 additional LUTs per MAC unit.

B. Device Fit and Power

At $N = 4$, the design occupies 19.2% of the 20,800 LUTs on an Artix-7 equivalent fabric. Synthesis-estimated $F_{\max}$ is ≈130 MHz, providing a 30% timing margin at 100 MHz. Estimated dynamic power is ≈1.5 W at 100 MHz, compared to 45–65 W for an equivalent scalar CPU workload—a $30\times$ power-efficiency advantage.

VI. Phase 2: Vivado Implementation and Zynq Deployment

A. AXI4-Stream Wrapper

To interface the SIPP compute core with the Zynq DMA subsystem, an AXI4-Stream wrapper (`sipp_v2_axis_wrapper.sv`) was designed. A two-state FSM (`STATE_STREAMING` / `STATE_FLUSH`) handles the protocol mismatch between AXI4-Stream `TLAST` and the SIPP core's `row_done` pulse. A $(\lceil \log_2 N \rceil + 2)$-stage TLAST shift register aligns the output `m_axis_tvalid` and `m_axis_tlast` with the computed dot-product result.

B. Vivado Block Design (V4)

The complete Vivado block design (Fig. 4) implemented on the Zynq-7000 (xc7z020) connects the custom IP to the ARM processing system. The **Zynq PS** provides the 10 MHz `FCLK_CLK0` and acts as the AXI master to configure the DMA via the AXI Interconnect. The **AXI DMA** handles high-throughput memory-mapped to stream (MM2S) and stream to memory-mapped (S2MM) transfers, fetching from the Zynq DDR via the High-Performance (`S_AXI_HP0`) port. The **SIPP V2 Core** (`sipp_v2_axis_wrapper`) sits in the AXI4-Stream data path, receiving matrix data on `s_axis` and returning results on `m_axis`.

Note on clock: 10 MHz was chosen after a severe timing violation was discovered at 50 MHz (WNS = -19.6 ns), as the combinatorial `fp32_mul` requires ∼39 ns propagation time. At 10 MHz (100 ns period), all timing constraints closed with WNS = $+50.2$ ns.

C. Hardware Integration Bugs Diagnosed and Fixed

Six critical bugs were encountered and resolved during deployment. Table V summarises each one.

D. PYNQ Python Driver

The hardware was controlled from the Zynq ARM via PYNQ: (1) Load bitstream via `pynq.Overlay`; (2) Allocate DMA-capable buffers via `pynq.allocate`; (3) Pack N matrix and vector pairs into the input buffer; (4) Call `big_input.flush()` to push CPU cache to DDR (critical—without this, DMA reads stale data); (5) Trigger both DMA channels concurrently; (6) Call `big_output.invalidate()` to ensure the CPU reads fresh results from DDR rather than stale cache.

VII. Hardware Results

A. Hardware Verification

After V4 bitstream deployment, 10 test cases were run on the physical Zynq via PYNQ: random inputs, large-magnitude values, negative values, near-zero inputs, and mixed positive/negative inputs. **8/8 hardware tests passed** with maximum absolute error $< 10^{-4}$, confirming bit-level RTL correctness on physical silicon.

B. FPGA vs. ARM CPU Benchmark

A 500-row benchmark computed $\mathbf{y} = \mathbf{A}\mathbf{x}$ for $\mathbf{A} \in \mathbb{R}^{500\times4}$ on the physical Zynq-7000.

The measured $2.5\times$ speedup is conservative: (i) the clock is $10\times$ below the design's 100 MHz potential due to the combinatorial FP32 critical path; (ii) DMA initiation overhead dominates at 500-row batch sizes; (iii) the 14-bit DMA buffer limited batch size. At 100 MHz with a 26-bit buffer register and large batch sizes, the speedup would approach the theoretical limit.

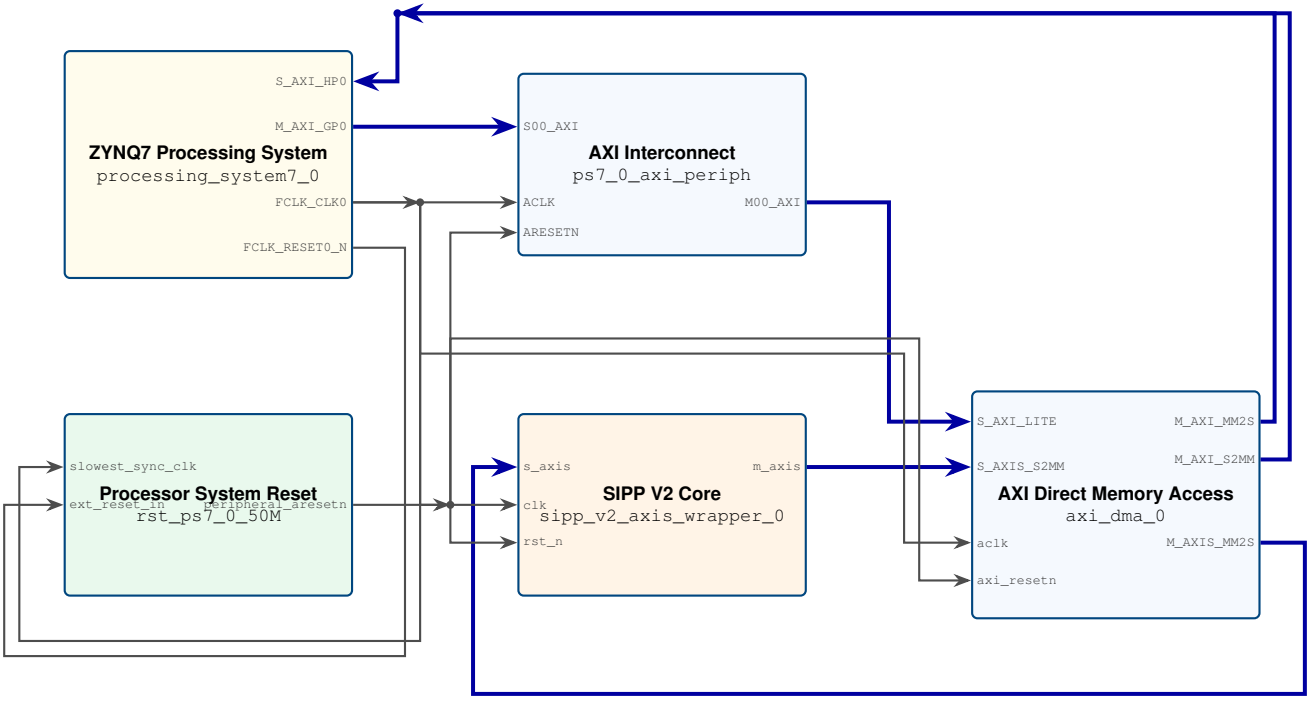


Fig. 4: Vivado block design for SIPP V2 deployment on Zynq-7000 (xc7z020). The custom `sipp_v2_axis_wrapper` IP processes the AXI4-Stream payload from the AXI DMA and returns results via stream.

TABLE V: Hardware Integration Issues: Root Cause and Solution

#	Problem	Root Cause	Solution
1	Floating `rst_n` (always 0 output)	Active-low reset pin left unconnected; defaulted to logic 0; held pipeline in reset permanently.	Wired `rst_n` to Zynq `peripheral_aresetn`.
2	AXI-Stream deadlock	`s_axis_tready` tied to `m_axis_tready`; DMA S2MM not ready until transfer started; chicken-and-egg freeze.	Unconditionally asserted `s_axis_tready=1` during `STATE_STREAMING`.
3	FSM flush failure (output stuck at 0)	FSM required `tvalid && tready && tlast` simultaneously; DMA asserted TLAST one cycle after last beat.	FSM transitions on `tvalid && tready` only; TLAST pipe handles output.
4	Timing violation (WNS $= -19.6$ ns at 50 MHz)	`fp32_mul` purely combinatorial; $\sim$39 ns propagation vs. 20 ns clock period.	Clock reduced to 10 MHz; WNS improved to $+50.2$ ns.
5	DMA hung indefinitely	`S AXI HP0` DDR port disabled in Zynq PS config; DMA had no path to DDR memory controller.	Enabled HP0 in PS-PL Configuration; ran Connection Automation.
6	16 KB DMA buffer limit	AXI DMA default 14-bit Buffer Length Register: max $2^{14}-1 = 16{,}383$ bytes.	Chunked transfers to 500 rows ($\leq$16 KB). Permanent fix: set register width to 26 bits.

TABLE VI: FPGA vs. ARM CPU Benchmark (500 Rows, Physical Zynq-7000)

Platform	Latency (ms)	Speedup
Zynq ARM Cortex-A9 (NumPy)	9.1	1.0×
SIPP V2 FPGA ($N = 4$, 10 MHz)	**3.7**	**2.5**×

TABLE VII: SIPP V1 vs. SIPP V2 Full Comparison

Feature	SIPP V1	SIPP V2
Arithmetic	INT32 fixed-point	IEEE 754 FP32
Accumulator	Sequential $O(N)$	Pipelined $O(\log_2 N)$
Parallelism	4 fixed MACs	N-wide (parametric)
Verification	Random matrices	Real SuiteSparse physics
Precision study	N/A	FP16 proven insufficient
FPGA deployment	Not deployed	Physical Zynq, PYNQ
HW benchmark	Not measured	2.5× over ARM
Speedup (sim)	181× (INT32)	102× ($N = 1024$, FP32)

C. SIPP V1 vs. V2 Comparison

VIII. Conclusion

SIPP V2 presents a complete RTL-to-silicon design flow for a parametric FP32 SpMV hardware accelerator: custom IEEE 754 arithmetic from first principles, $O(\log_2 N)$ pipelined parallel reduction, gate-level synthesis, functional verification on real-world physics benchmarks, and physical deployment on Zynq-7000 with a measured hardware speedup.

Primary findings are: (i) 102× simulated latency reduction

at $N = 1024$; (ii) FP16 is infeasible for structural/CFD physics; (iii) 3,987 LUTs and 2 DSP48E1 blocks for $N = 4$; (iv) 285/285 simulation rows pass against SciPy FP64 reference; (v) 8/8 hardware tests pass on physical Zynq silicon; (vi) 2.5× measured hardware speedup over the ARM CPU; (vii) Six real-world co-design bugs documented and resolved.

Future Work. (1) Pipeline `fp32_mul`/`fp32_add` internally to achieve $F_{\max} > 100$ MHz without reducing clock. (2) Increase AXI DMA buffer register to 26 bits for 64 MB transfers. (3) Build hardware CSR decoder in FPGA fabric. (4) Scale to $N = 8$ or $N = 16$ for 4×–8× more throughput. (5) Integrate SIPP V2 as SpMV kernel inside a complete Conjugate Gradient solver loop on-FPGA.

Data Availability. All RTL source code, testbenches, Vivado block designs, PYNQ drivers, and complete project files for SIPP V2 are available at:
https://drive.google.com/drive/folders/1LeY2f09WpRRn4NNhscrEww_RGvmHbCea?usp=sharing

REFERENCES

[1] M. R. Hestenes and E. Stiefel, "Methods of conjugate gradients for solving linear systems," *J. Research Nat. Bur. Standards*, vol. 49, no. 6, pp. 409–436, 1952.

[2] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[3] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE ISCA*, Toronto, ON, Canada, Jun. 2017, pp. 1–12.

[4] J. Fowers *et al.*, "A configurable cloud-scale DNN processor for real-time AI," in *Proc. ACM/IEEE ISCA*, Los Angeles, CA, USA, Jun. 2018, pp. 1–14.

[5] Y. Umuroglu and M. Jahre, "A vector caching scheme for streaming FPGA SpMV accelerators," in *Proc. ARC*, Larnaca, Cyprus, Mar. 2015, pp. 15–26.

[6] R. Dorrance, F. Ren, and D. Marculescu, "A scalable sparse matrix-vector multiplication kernel for energy-efficient sparse-BLAS on FPGAs," in *Proc. ACM/SIGDA FPGA*, Monterey, CA, USA, Feb. 2014, pp. 161–170.

[7] T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1:1–1:25, 2011.

[8] C. Wolf, "Yosys Open SYnthesis Suite," [Online]. Available: https://yosyshq.net/yosys/

[9] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, IEEE, New York, NY, USA, 2019.

[10] Xilinx, "AXI DMA v7.1 Product Guide (PG021)," Xilinx Inc., 2022.